

Critical Evaluation of AutoML-Based Intrusion Detection for Zero-Touch Network Security

A Reproduction and Methodological Evaluation of “Enabling AutoML for Zero-Touch Network Security: Use-Case Driven Analysis”

Ameer Rohana
Student ID: 215631375
Data Science in Cyber
University of Haifa

July 2026

Executive Summary

This project critically evaluates the intrusion-detection case study presented in the paper *Enabling AutoML for Zero-Touch Network Security: Use-Case Driven Analysis*. The source proposes an automated machine-learning workflow for developing an intrusion detection system and also discusses the vulnerability of that system to adversarial attacks.

The study used a prepared sample of the CICIDS2017 network-traffic dataset. The source’s clean intrusion detection system was based on LightGBM and was reported to achieve accuracy, precision, recall, and F1 score above 99%.

The first stage of this project reproduced the source’s clean-data pipeline as closely as possible. The reproduced results were very close to those reported by the authors. The largest difference among the four primary metrics was only 0.0884 percentage points. This supports the narrow claim that the published clean-data result can be reproduced when the original procedure is followed.

Reproduction alone, however, does not establish that the methodology is unbiased. The original workflow performed several operations before or with knowledge of the train-test split, including normalization, feature selection, and parts of model selection. In addition, identical predictor profiles could appear on both sides of the split. These choices create a risk of optimistic performance estimates.

A corrected protocol was therefore developed. Identical feature profiles were grouped so that they could not cross the training, validation, and final-test boundaries. All feature, model, and threshold decisions were made using development data only. The final test set was then evaluated exactly once.

Four main model families were compared: Logistic Regression, Random Forest, Histogram Gradient Boosting, and LightGBM. LightGBM provided the best overall balance of attack recall, benign-traffic specificity, MCC, balanced accuracy, and total error count. The final candidate used the original 19 numerical predictors and a validation-selected threshold of 0.546.

On the untouched corrected final test set, the model produced 4,496 true negatives, 9 false positives, 5 false negatives, and 1,157 true positives. Its accuracy was approximately 99.75%, with 99.23% precision, 99.57% recall, 99.80% specificity, and a 99.40% F1 score.

The results show that the source’s clean-data performance claim is genuine and not explained entirely by the identified methodological weaknesses. Nevertheless, the evidence is not sufficient to establish real-world deployment readiness. The experiment uses one sampled dataset, lacks reliable chronological information, and does not test generalization to new organizations or attack families. The original

adversarial attack and defense claims were also not fully reproduced in the completed experimental scope.

The final conclusion is therefore positive but qualified: the approach is a strong research baseline for flow-based intrusion detection, but it requires stricter validation, richer network context, drift monitoring, and layered security controls before operational deployment.

1 Summary of the Source

1.1 Problem and Motivation

The selected source investigates cybersecurity in zero-touch networks. Such networks aim to automate configuration, optimization, monitoring, and recovery with minimal human intervention. Machine learning can support this automation, but it also introduces two related risks.

First, building effective machine-learning security systems normally requires expertise in preprocessing, feature selection, model choice, and hyperparameter tuning. Second, once a network relies on an ML-based security component, an attacker may attempt to manipulate its inputs and cause incorrect decisions.

The paper therefore studies both the automatic construction of an intrusion detection system and the security of the resulting model against adversarial manipulation [1].

1.2 Dataset and Proposed Method

The relevant implementation uses a prepared sample of CICIDS2017, a network-intrusion dataset containing benign flows and several forms of malicious traffic [3].

The source workflow includes:

1. automated preprocessing and normalization;
2. feature engineering and feature selection;
3. class balancing using SMOTE;
4. training an optimized LightGBM intrusion detector;
5. generating adversarial samples using DTA, FGSM, and BIM;
6. training a detector to distinguish original and adversarial samples; and
7. filtering suspicious samples and retraining the IDS.

The authors report that the clean LightGBM model achieves approximately 99.82% accuracy and 99.56% F1 score. They further report that adversarial attacks can reduce IDS performance sharply, while their detection-and-filtering procedure restores performance close to the clean baseline.

This project treats those results as claims to investigate rather than as conclusions to accept automatically.

2 Critical Evaluation

2.1 What Was Supported

The clean-data LightGBM result was reproduced successfully. Under a close reproduction of the source procedure, the four main metrics were within 0.10 percentage points of the authors' executed notebook output.

This is important because it shows that the published clean result was not merely descriptive or impossible to obtain. The repository, prepared dataset, selected features, and model configuration contained enough information to reconstruct the main clean IDS experiment.

The corrected evaluation also remained extremely strong. Therefore, the predictive signal in this dataset appears genuine. The model was not successful only because of one particular random split or one isolated implementation detail.

2.2 Methodological Weaknesses

The main concerns relate to how the original performance was estimated.

Normalization and feature selection were performed using information from the complete dataset rather than being fitted only inside the training data. Hyperparameter evidence also included repeated access to the test result. These decisions allow information about the evaluation data to influence development.

A second concern is duplicate-related leakage. Some observations have identical values across all predictor columns. A conventional random split may place the same predictor profile in both the training and test sets. Even when the rows are technically different, the evaluation no longer represents a completely unseen pattern.

The source also does not provide external validation on another organization, a later capture period, or unseen attack families. Strong performance on one prepared sample therefore does not establish long-term stability or deployment readiness.

Finally, the source’s adversarial claims are broader than the clean IDS claim. The current project successfully reproduced the clean branch, but it did not complete an independent reproduction of every DTA, FGSM, BIM, adversarial-detection, and filtering experiment. Those claims cannot be confirmed solely from the clean-data results.

2.3 Overall Judgment

The source’s narrow claim is supported:

A LightGBM-based IDS can achieve very high classification performance on the supplied CICIDS2017 sample.

Its broader claim is only partially supported:

The available evidence is insufficient to prove unbiased generalization, adversarial robustness, or readiness for autonomous deployment in other network environments.

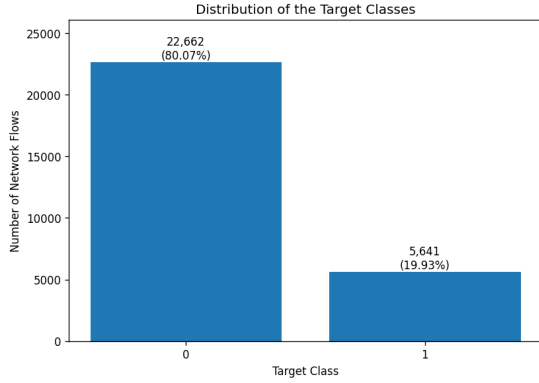
The corrected experiment strengthens the clean predictive conclusion while narrowing its interpretation.

3 Feature Engineering Analysis

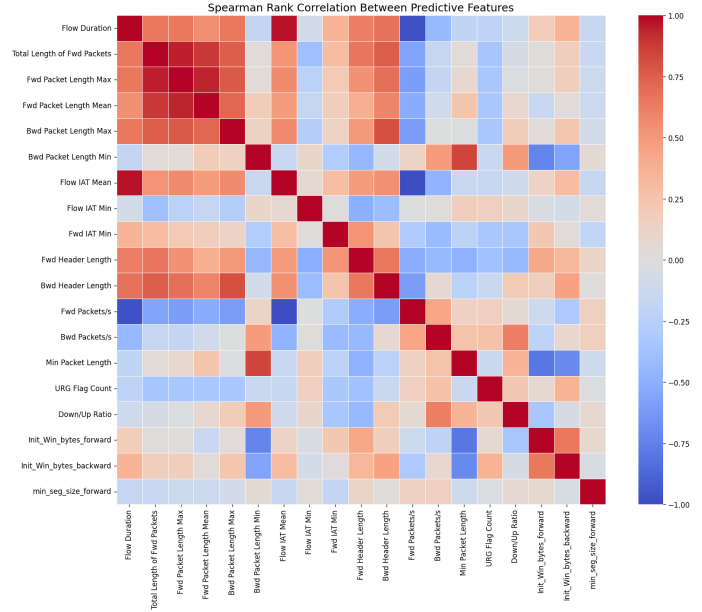
3.1 Data Characteristics

The working dataset contained 28,303 observations, including 22,662 benign flows and 5,641 attack flows. Attacks therefore represented approximately 19.93% of the supplied data.

All retained predictors were numerical flow-level measurements. They described packet sizes, rates, durations, inter-arrival times, header lengths, flags, window values, and segment properties.



(a) Target distribution from Notebook Section 4.1.



(b) Spearman feature correlations from Notebook Section 4.9.

Figure 1: The dataset is imbalanced, and several predictors have strong monotonic relationships. The correlation analysis was used to identify possible redundancy without assuming normal distributions or purely linear relationships.

Spearman correlation was emphasized because many network-flow features were skewed and contained extreme values. Unlike Pearson correlation, Spearman correlation measures monotonic rank relationships and is less sensitive to large numerical outliers.

Several strong relationships were found among packet-size, timing, and header-related variables. These relationships do not automatically make a feature useless, because tree-based models can still benefit from interactions between correlated variables. They do, however, indicate that feature-selection decisions should be made inside the development data rather than on the complete dataset.

3.2 Original and Corrected Feature Engineering

The source normalized the numerical variables, selected 12 predictors using feature importance and correlation rules, and applied SMOTE to the training data.

The corrected workflow considered scaling, transformations, feature creation, redundancy removal, feature selection, dimensionality reduction, and imbalance handling. These alternatives were evaluated using training and validation evidence only.

The final LightGBM pipeline retained the original 19 predictors without scaling, feature selection, dimensionality reduction, or synthetic oversampling. This was not an assumption that feature engineering is unnecessary in general. It was an empirical conclusion: the additional representations did not provide enough validated improvement to justify their extra complexity.

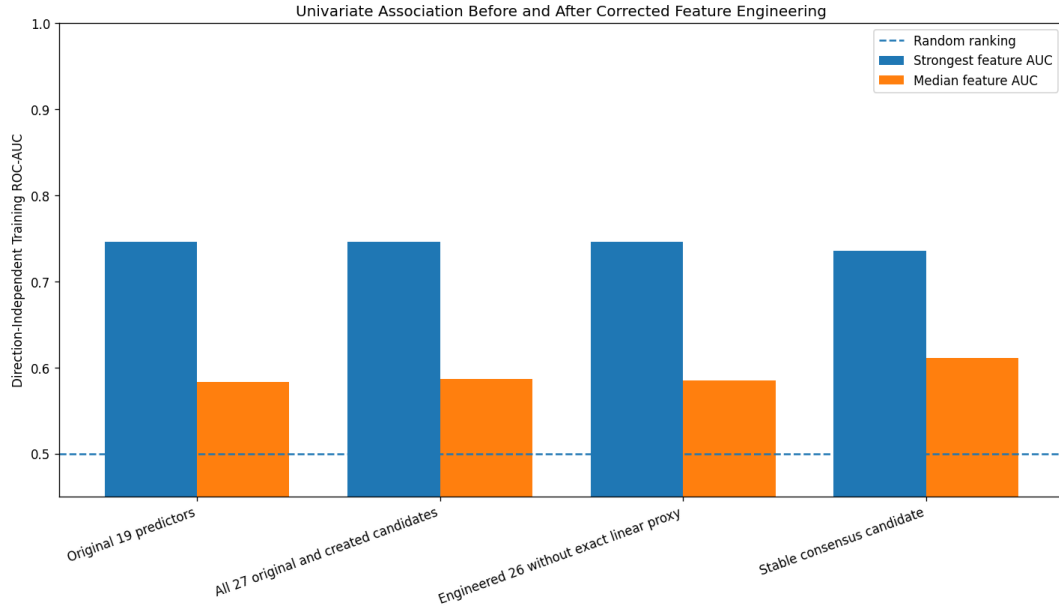


Figure 2: Univariate association before and after the evaluated feature engineering strategies. Source: Notebook Section 7.13. Created features were examined, but the simpler 19-feature representation was retained for the final pipeline.

From a cybersecurity perspective, the largest missing information is not another mathematical transformation of the same values. More useful future features may include protocol, source and destination port, host role, application, repeated connection behavior, neighboring flows, and temporal history. These could separate legitimate and malicious flows that have similar aggregate packet statistics.

4 Reproducibility Analysis

The source repository, selected notebook, dataset, and main model configuration were accessible [2]. However, the original notebook could not be executed unchanged in the current environment.

The source used an older Python and library environment. Several interfaces required compatibility changes, including:

- replacing `SMOTE.fit_sample` with `fit_resample`;
- removing the obsolete SMOTE `n_jobs` argument;
- replacing the deprecated `np.bool` alias;
- replacing removed uses of `DataFrame.append`; and
- updating legacy Adversarial Robustness Toolbox wrappers.

These changes were treated as interface modifications rather than methodological improvements. Verification checks confirmed that the clean reproduction still used the same normalization decision, the same 12 selected features, the same random split, the same balancing behavior, and the same LightGBM configuration.

The clean IDS branch was highly reproducible. The remaining small numerical differences can reasonably result from library versions, internal model randomness, SMOTE randomness, and different Colab hardware.

The complete attack-and-defense pipeline was less reproducible. Current ART and TensorFlow interfaces differ substantially from those used in the source notebook, and the completed project did not independently execute every adversarial and recovery branch. The appropriate conclusion is therefore that the clean IDS result is reproducible, while the complete adversarial pipeline remains only partially assessed.

5 Experimental Results

5.1 Corrected Experimental Design

The corrected dataset was divided into:

- 16,851 training observations;
- 5,785 independent validation observations; and
- 5,667 final-test observations.

Rows with identical predictor profiles were assigned to the same group. The split was then performed at group level, preventing an identical feature profile from crossing subset boundaries.

Five-fold group-aware cross-validation was used during development. The independent validation set was used for the final pipeline decision and threshold selection. The final test set remained untouched until the model family, feature representation, and threshold had been frozen.

5.2 Models and Metrics

The controlled comparison included:

- a majority-class baseline;
- Logistic Regression;
- Random Forest;
- Histogram Gradient Boosting; and
- LightGBM.

Accuracy was included for completeness, but it was not used alone. The main evaluation also considered precision, recall, specificity, F1 score, F2 score, balanced accuracy, Matthews correlation coefficient (MCC), ROC-AUC, average precision, confusion counts, and computational cost.

Recall measures the proportion of attacks detected:

$$\text{Recall} = \frac{TP}{TP + FN}.$$

Specificity measures the proportion of benign flows correctly rejected:

$$\text{Specificity} = \frac{TN}{TN + FP}.$$

Precision measures the proportion of generated alerts that correspond to actual attacks:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

These quantities have direct operational meaning. False positives consume investigation time, while false negatives may allow attacks to continue undetected.

5.3 Cross-Validation Comparison

LightGBM was the strongest development model. It achieved the highest balanced accuracy and MCC and produced the smallest pooled out-of-fold error count.

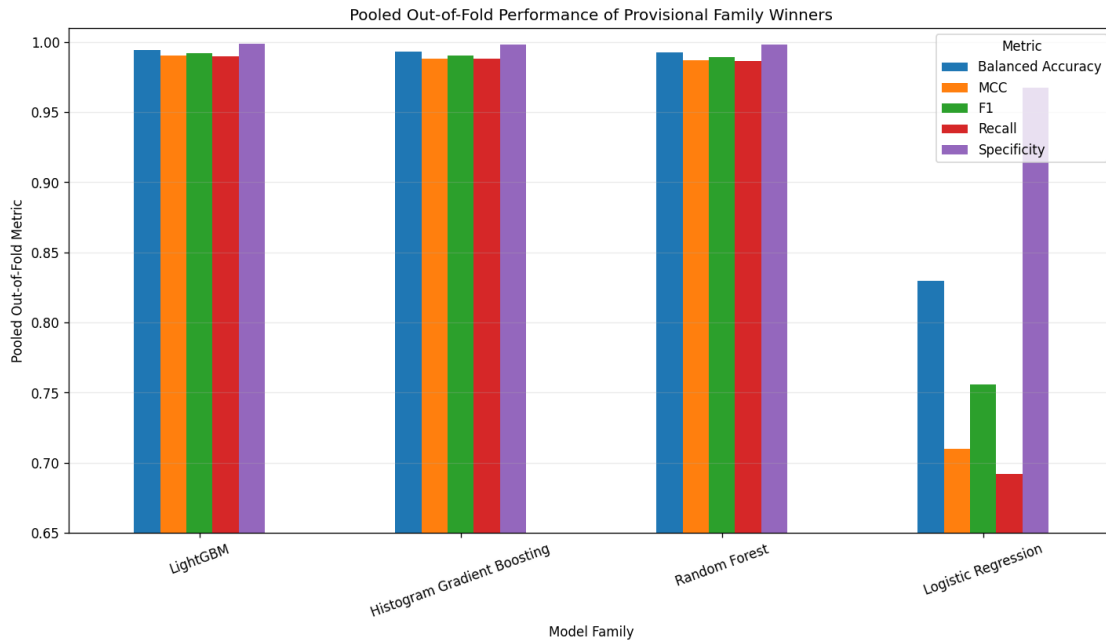


Figure 3: Pooled out-of-fold comparison of the provisional family winners. Source: Notebook Section 8.8.

Histogram Gradient Boosting and Random Forest also performed strongly, which indicates that the data favor nonlinear tree-based decision boundaries. Logistic Regression was weaker, suggesting that the class structure is not captured as effectively by one linear boundary.

5.4 Final Test Results

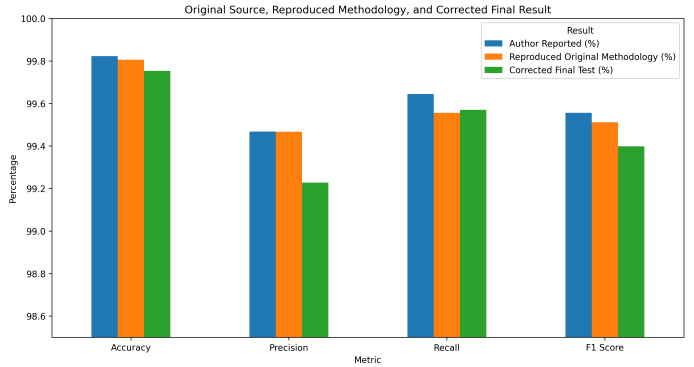
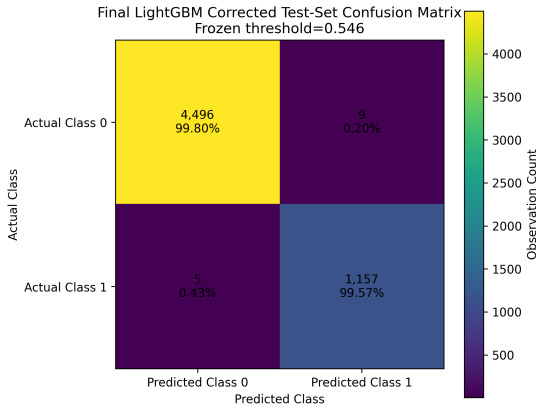
The final selected candidate was `LGBM_00_initial`, using all 19 predictors and a frozen classification threshold of:

$$t = 0.546.$$

The final test set was evaluated exactly once.

Table 1: Final performance of the frozen LightGBM pipeline. Values are taken from Notebook Section 9.11.

Metric	Final-Test Result
Accuracy	99.7530%
Precision	99.2281%
Recall	99.5697%
Specificity	99.8002%
F1 score	99.3986%
Balanced accuracy	99.6850%
MCC	99.2434%
ROC-AUC	99.9861%
Average precision	99.9492%
True negatives	4,496
False positives	9
False negatives	5
True positives	1,157
Total errors	14



(a) Corrected final-test confusion matrix.

(b) Source, reproduction, and corrected results.

Figure 4: Final test behavior and comparison with the original source. Sources: Notebook Sections 9.11 and 9.13.

The corrected accuracy was slightly below the source value, but the difference was small. The stricter split and evaluation rules therefore did not remove the model’s strong predictive performance.

Table 2: Comparison of source-reported, reproduced, and corrected primary metrics. Source: Notebook Section 9.13.

Metric	Source	Reproduction	Corrected
Accuracy	99.8230%	99.8057%	99.7530%
Precision	99.4680%	99.4671%	99.2281%
Recall	99.6440%	99.5556%	99.5697%
F1 score	99.5560%	99.5113%	99.3986%

The reproduction differed from the source by at most 0.0884 percentage points. The corrected result

differed by at most 0.2399 percentage points, with precision showing the largest reduction.

These values are descriptively close, but they are not perfectly equivalent experiments. The source used a different split, feature representation, threshold, and development policy.

5.5 Error and Threshold Analysis

The final model made only 14 errors, but the failures were not all borderline cases. Only one error was within 0.05 of the frozen threshold, while 11 were more than 0.25 away from it.

Packet-size, header, segment-size, and timing variables appeared repeatedly among the misleading features. In particular, `Fwd Packet Length Mean` and `min_seg_size_forward` were associated with many errors.

Three false-positive observations shared exactly the same values across all 19 predictors and received the same model score. This indicates a repeated ambiguous behavior rather than three completely independent mistakes.

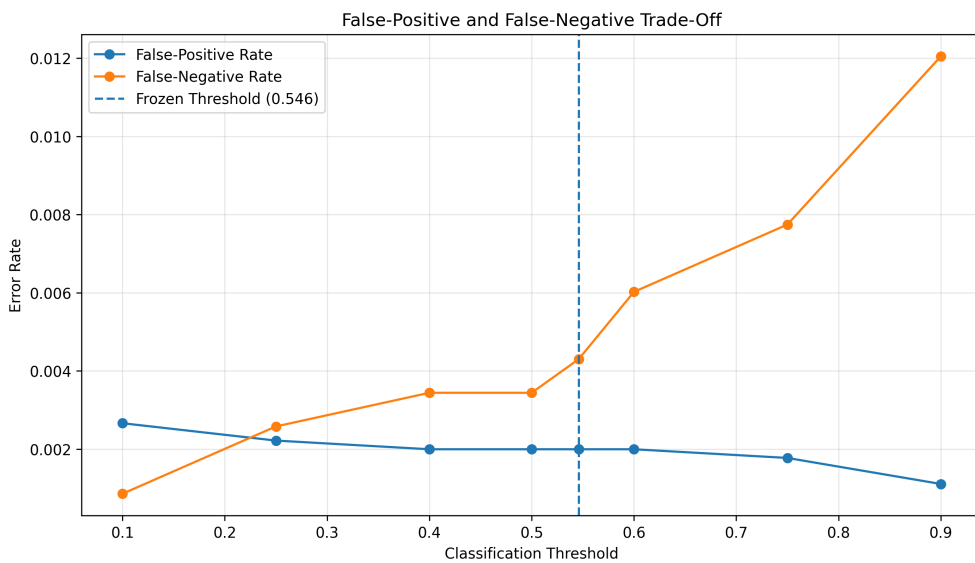


Figure 5: Descriptive false-positive and false-negative trade-off. Source: Notebook Section 10.8. The alternative thresholds were analyzed after evaluation and were not used to replace the frozen threshold of 0.546.

Lower thresholds recovered some missed attacks with a small increase in false alerts. Higher thresholds reduced few false positives but caused a larger increase in missed attacks.

It would be invalid to choose a new threshold simply because it looked better on the final test set. The figure is therefore a sensitivity analysis, not an additional model-selection step.

At the frozen operating point, the observed error rates corresponded to approximately:

- 19.98 false alerts per 10,000 benign flows; and
- 43.03 missed detections per 10,000 attacks.

The false-negative count is smaller in absolute terms, but missed attacks will often have a greater security cost than false alerts. Threshold policy should therefore depend on analyst capacity, attack severity, and the organization's risk tolerance.

6 Conclusions

6.1 Main Findings

The original clean LightGBM result was successfully reproduced. The corrected evaluation also maintained very high performance, showing that the source’s central predictive finding is supported on the supplied dataset.

The most important contribution of the corrected study was not a large increase in accuracy. It was the introduction of a more trustworthy evaluation procedure:

- identical predictor profiles could not cross split boundaries;
- preprocessing and model decisions used development data only;
- threshold selection used validation data;
- the final test set was evaluated once; and
- every final result was accompanied by integrity checks.

6.2 Strengths of the Original Work

The source addresses an important security problem, provides public code, uses interpretable flow-level features, and selects a model family that is well suited to tabular network data.

Its clean-data result was not only strong but reproducible. The paper and repository also provide a useful starting point for discussing how adversarial machine learning may affect automated network security.

6.3 Limitations

The original evaluation contains risks of preprocessing leakage, feature-selection leakage, duplicate-profile overlap, and test-based model development.

The supplied data also lack reliable absolute timestamps and several forms of network context. As a result, the study cannot establish resistance to concept drift or generalization across organizations, hosts, protocols, and future attacks.

The current project did not complete an independent evaluation of all the original adversarial attack-and-defense branches. The conclusions about DTA, FGSM, BIM, adversarial detection, and recovery should therefore remain separate from the supported clean-data conclusion.

6.4 Recommendations

Future work should:

- validate the model on external and later-period datasets;
- add protocol, port, host, application, and sequence context;
- investigate repeated false-positive profiles;
- assess model-score calibration;
- monitor concept drift;
- evaluate realistic and constrained adversarial attacks; and
- combine ML predictions with rules, endpoint evidence, and analyst review.

A simple, well-controlled pipeline is preferable to a more complicated system evaluated with weak data separation.

7 Summing It Up

This project examined whether the claims of a published AutoML-based network intrusion detection study were supported by reproducible and methodologically sound evidence.

The clean IDS result was reproduced closely. A corrected group-aware evaluation then showed that LightGBM still achieved approximately 99.75% accuracy, 99.57% recall, and 99.23% precision on an untouched final test set.

Therefore, the source’s main clean-data claim is supported. The selected features contain strong predictive information, and LightGBM is an effective model for this particular flow-based classification problem.

The broader claims require greater caution. One sampled dataset cannot prove operational reliability, future generalization, adversarial robustness, or safe zero-touch deployment. The model also made several high-confidence errors that are unlikely to be solved by a minor threshold adjustment alone.

The approach is recommended as a useful research baseline and as one possible component of a layered intrusion detection system. It is not recommended as a standalone production security control without external validation, richer context, drift monitoring, calibrated confidence, and human oversight.

The main lesson is that trustworthy cybersecurity machine learning depends not only on high scores, but also on leakage prevention, protected evaluation, meaningful security metrics, detailed failure analysis, and careful limits on the conclusions drawn from the data.

References

- [1] L. Yang, M. El Rajab, A. Shami, and S. Muhaidat, “Enabling AutoML for Zero-Touch Network Security: Use-Case Driven Analysis,” *IEEE Transactions on Network and Service Management*, vol. 21, no. 3, pp. 3555–3582, June 2024. doi: [10.1109/TNSM.2024.3376631](https://doi.org/10.1109/TNSM.2024.3376631) .
- [2] Western OC2 Lab, “AutoML and Adversarial Attack Defense for Zero-Touch Network Security,” GitHub repository. Available: <https://github.com/Western-OC2-Lab/AutoML-and-Adversarial-Attack-Defense-for-Zero-Touch-Network-Security>.
- [3] Canadian Institute for Cybersecurity, “Intrusion Detection Evaluation Dataset, CICIDS2017.” Available: <https://www.unb.ca/cic/datasets/ids-2017.html>.